ENSE701 Contemporary Issues in Software Engineering

Assignment 1: Software Engineering Investigation and Project Planning

DueMate: a small deadline tracker for students

Haoguang Wang

Student ID 25305403

2 November 2026

Individual submission, approved by the course lecturer

Word count (main text, tables, citations, contribution statement): 1499

1. Project definition

Students keep deadlines in four places: the learning management system, an email inbox, lecture slides and memory. The information exists but is not comparable, so the question that shapes a week, what is due next, takes minutes to reconstruct. DueMate answers it in one screen: a single list of assignments with course, title, due date, weight and status, ordered so that overdue work comes first. The value is a lower chance of missing a weighted deadline, and a visible answer to where effort belongs.

The primary stakeholder is one student who owns the data and runs the tool locally. The lecturer is an indirect stakeholder, since the tool protects assessment outcomes, and the assessor is an unusual one: the software counts as working only if somebody else can start it and reproduce the tests, which makes reproducibility a design constraint. Scope covers create, read, update and delete for assignments, urgency ordering, a status filter and an iCalendar export so reminders come from a calendar the user already trusts. Authentication, multi-user sharing, notifications and analytics are excluded, as each adds a dependency a single user does not need.

Constraints are one-week iterations, an individual submission, zero budget, no external service and a two-week deadline. Three engineering issues follow. Scope creep is the likeliest cause of a half-finished increment, because the domain invites growth towards reminders. Coordination and peer review, which a team would supply, are absent, so review must be engineered as a checklist activity (Fagan, 1976). Verification is asymmetric: automated API tests cannot see a broken form or an invisible warning colour (Jureczko et al., 2020), and time-dependent deadline logic threatens determinism unless the clock is controlled.

2. Comparison and selection of development approaches

Three approaches were compared on response to change, feedback speed, traceable evidence, coordination overhead and fitness for one-week cycles (Table 1).

Criterion	Plan-driven	Iterative and incremental (selected)	Agile framework
Changing requirements	Expensive; baseline fixed early	Absorbed at iteration boundary	Welcomed; costly to re-plan
Feedback speed	One late integration point	Runnable increment each week	Continuous, via team rituals
Traceable evidence	Heavy to produce	Backlog, tests, commits per story	Strong if artefacts maintained
Coordination overhead	High: documents, sign-off	Low: one backlog, one board	High: roles and ceremonies
Fit for one-week cycles	Poor: no software until the end	Good: goals sized to a week	Poor: overhead exceeds build time

Table 1. Comparison of development approaches against the selection criteria.

Plan-driven development suits stable requirements and formal handover, but concentrates feedback at the end, when no time remains to react, and even sequentially presented projects relied on iteration in practice (Larman & Basili, 2003). A full agile framework is the opposite problem: its ceremonies are repaid by team coordination, which a single developer does not have. Iterative and incremental development sits between the two, delivering a runnable increment each cycle, and its risk-driven variant builds the riskiest element first (Boehm, 1988), which here is the deadline logic.

The selection is iterative and incremental with a small set of agile practices retained where evidence is strong: a prioritised backlog with testable criteria, tests before implementation, continuous integration and a retrospective that changes one thing at a time. Test-first development is kept because controlled studies report better design quality and fewer defects, subject to discipline rather than the label (Janzen & Saiedian, 2005, 2008); continuous integration because earlier defect detection is a problem a solo developer still has (Hilton et al., 2016). Team-only ceremonies are dropped.

The choice is context-dependent. In a distributed team it still fits, but coordination artefacts must become explicit: written acceptance criteria, asynchronous review and small integrated changes, because dispersion degrades informal communication (Anh et al., 2012; Camara & Marinho, 2024). In a regulated system the iteration would sit inside traceability and gate reviews, since agile practices can be tailored to such

environments when documentation is treated as part of the process rather than overhead (Fitzgerald et al., 2013). The plan's shape survives in both; what changes is how much is written down.

3. Project plan

The backlog is prioritised with MoSCoW and every story carries given/when/then acceptance criteria, so a story is done only when a named test passes. Table 2 shows the ten stories; full criteria are in docs/backlog.md. Seven were committed.

ID	Story (abbreviated)	Priority	Iteration
US-01	Add an assignment	Must	1
US-02	Order by deadline	Must	1
US-03	Validation messages	Must	1
US-05	Mark done, reopen, delete	Must	1
US-06	Filter by status	Should	1
US-07	Export iCalendar	Should	1
US-04	Prioritise overdue work	Should	2 (trial change)
US-08	Edit a single field	Could	2, not delivered
US-09	Search by course code	Could	2, not delivered

Table 2. Prioritised backlog (abbreviated; acceptance criteria in docs/backlog.md).

Iteration 1 targets capture and review: adding assignments, ordered listing, validation, status changes, filtering and calendar export. Iteration 2 targets risk visibility: overdue and soon-due work classified, sorted first and styled, plus one tracked, reviewed and tested change. Table 3 records goals and outcomes.

Iteration	Goal	Committed items	Outcome
1	Record assignments and see what is coming	US-01, US-02, US-03, US-05, US-06, US-07	Delivered; 13 tests; every criterion covered
2	Make deadline risk visible; prove the workflow	US-04, CI workflow, four retrospective actions	Delivered; red 3/7, then 16/16 green; one defect fixed in review; US-08/09 deferred

Table 3. Iteration goals, committed items and outcomes.

The risk register was reviewed at the start of each iteration; Table 4 shows the four risks that could stop the work. Two deserve naming. The untested browser layer is accepted as debt with a plan for Assignment 2, because unmanaged technical debt compounds and becomes invisible to the next reader (Li et al., 2015). The missing second reader is met by a checklist-based self-review with findings recorded before merge (Fagan, 1976).

Risk	Likelihood / impact	Response
R1 Scope creep towards reminders	High / High	MoSCoW fixed; new ideas go to out-of-scope
R2 Requirements change mid-iteration	Medium / Medium	Change accepted only into the next iteration
R3 Untested browser layer	High / Medium	Manual script each iteration; DOM test in Assignment 2
R6 Solo review may miss defects	High / Medium	Checklist self-review with findings recorded before merge

Table 4. Key risks and responses.

Figure 1 shows the structure: a dependency-free browser client over a JSON API, a route handler owning HTTP concerns, a domain module holding validation, urgency and ordering as pure functions, an export module for iCalendar, and one persistence module. Four design decisions carried the most weight. Zero third-party dependencies mean the assessor runs and tests the software with no install step. The injected clock keeps overdue tests deterministic. Derived values are computed on the server, so browser and export cannot disagree about what overdue means. The JSON file store trades concurrency for a setup with no database, which one user does not need.

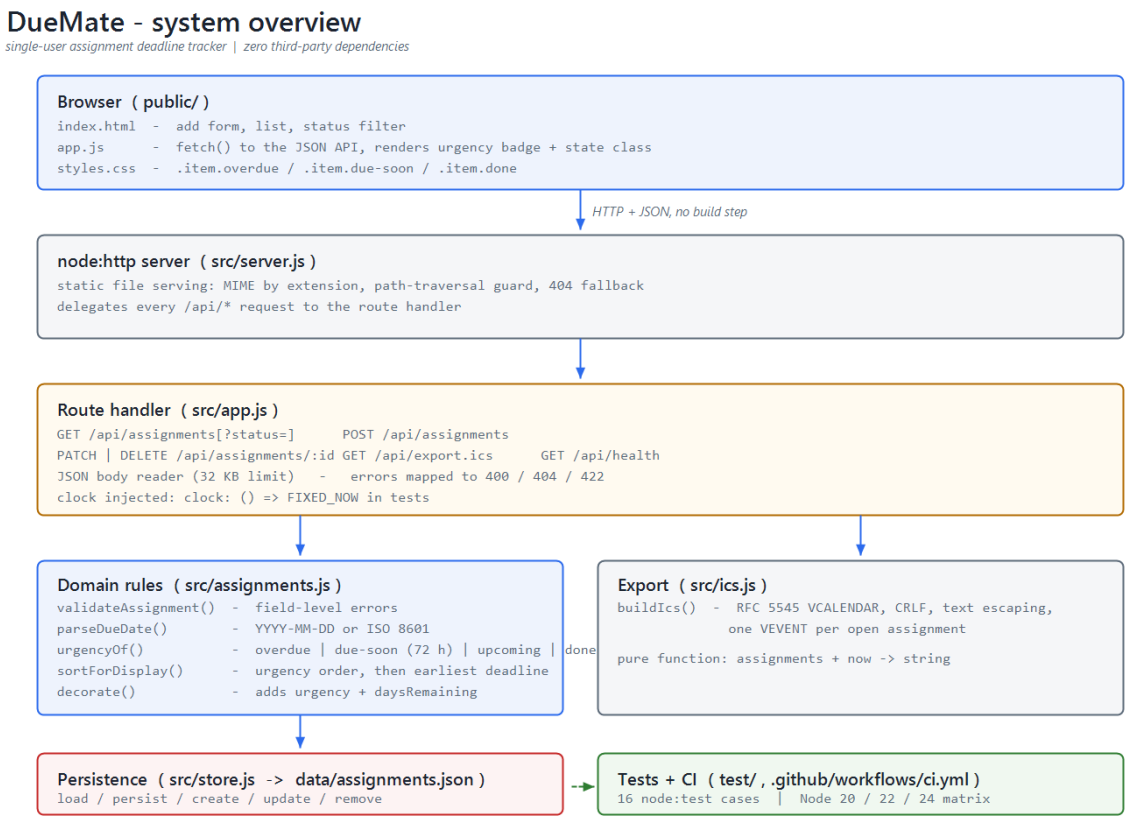


Figure 1. System overview of DueMate, showing the request path from browser to persistence and the modules covered by automated tests.

4. Workflow trial and evidence

The workflow was trialled with one small change, story US-04: overdue and soon-due work classified, sorted first and made visually distinct. Work began on branch feature/us-04-urgency with the acceptance tests written before implementation. That run failed as intended, three passing and four failing. The implementation added an urgency classifier, an ordering function and a badge with state styling, after which sixteen of sixteen tests passed; the change touches three files and adds sixty-six lines.

Review was a structured self-review against the acceptance criteria. Five findings were recorded and three fixed before merge: day rounding reported zero days for an item twelve hours late while labelling it overdue, the export test named a stale story identifier, and the README omitted the derived fields. Two were accepted as debt: a repeated urgency computation in the comparator and the missing browser test. Traceability was updated in the same commit, and CI runs the suite on Node 20, 22 and 24 (Table 5).

Evidence	Location	What it shows
Backlog with acceptance criteria	docs/backlog.md	Stories testable before code exists
Tracked requirement to merge	branch feature/us-04-urgency; 4a912a1, b846459	Plan to release for one requirement
Red then green runs	docs/evidence/iteration-2/	Tests preceded code; 16/16 after
Review findings and traceability	docs/review-record.md; docs/traceability.md	Five findings, three fixed; requirement to test mapping

Table 5. Evidence index for the workflow trial.

Two lessons followed. Tests written first changed the design, not only the confidence: the rounding defect surfaced while specifying behaviour. And documentation drift was the most frequent defect: two of five findings were stale statements, suggesting prose needs the same review as code.

5. Contribution statement

The lecturer approved an individual submission, so the roles were held sequentially: analyst for the problem definition, backlog and risk register; designer for the architecture; developer for the implementation including the trial change; reviewer for the self-review and its five findings; tester for the sixteen automated tests and the CI matrix. Per-role evidence links are in docs/contribution.md and docs/evidence-index.md. No third-party source code is reused: the application uses Node.js built-in modules only.

References

Anh, N.-D., Cruzes, D. S., & Conradi, R. (2012). Dispersion, coordination and performance in global software teams: A systematic review. In Proceedings of the ACM-IEEE International Symposium on Empirical Software Engineering and Measurement (pp. 129–138). ACM. https://doi.org/10.1145/2372251.2372274

Boehm, B. W. (1988). A spiral model of software development and enhancement. Computer, 21(5), 61–72. https://doi.org/10.1109/2.59

Camara, R., & Marinho, M. (2024). Agile tailoring in distributed large-scale environments using agile frameworks: A systematic literature review. CLEI Electronic Journal, 27(1). https://doi.org/10.19153/cleiej.27.1.8

Fagan, M. E. (1976). Design and code inspections to reduce errors in program development. IBM Systems Journal, 15(3), 182–211. https://doi.org/10.1147/sj.153.0182

Fitzgerald, B., Stol, K.-J., & O'Sullivan, R. (2013). Scaling agile methods to regulated environments: An industry case study. In Proceedings of the 35th International Conference on Software Engineering (pp. 863–872). IEEE. https://doi.org/10.1109/ICSE.2013.6606635

Hilton, M., Tunnell, T., Huang, K., Marinov, D., & Dig, D. (2016). Usage, costs, and benefits of continuous integration in open-source projects. In Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (pp. 426–437). ACM. https://doi.org/10.1145/2970276.2970358

Janzen, D., & Saiedian, H. (2005). Test-driven development: Concepts, taxonomy, and future direction. Computer, 38(9), 43–50. https://doi.org/10.1109/MC.2005.314

Janzen, D., & Saiedian, H. (2008). Does test-driven development really improve software design quality? IEEE Software, 25(2), 77–84. https://doi.org/10.1109/MS.2008.34

Jureczko, M., Kajda, Ł., & Górecki, P. (2020). Code review effectiveness: An empirical study on selected factors influence. IET Software, 14(7), 794–805. https://doi.org/10.1049/iet-sen.2020.0134

Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. Computer, 36(6), 47–56. https://doi.org/10.1109/MC.2003.1204375

Li, Z., Avgeriou, P., & Liang, P. (2015). A systematic mapping study on technical debt and its management. Journal of Systems and Software, 101, 193–220. https://doi.org/10.1016/j.jss.2014.12.027

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.